Doomscroll

AI-Based Project Initialisation — Lab 3

Version 1.0
Prepared by: Project Team -- SC2006
Date: 6 November 2026

This report documents two AI consultations carried out per Lab 3 manual section 3.3: a repeated technology-stack recommendation prompt (3.3.1-3.3.2), using the team's current class diagram instead of Lab 2's, and an AI-generated minimal application skeleton (3.3.3-3.3.4). Both exercises were run as fresh, standalone AI conversations with no access to the team's actual codebase, conversation history, or prior reports -- each was given only the materials quoted below, matching how a student would open a new chat and paste in the class diagram. Prompts and AI responses are reproduced in full and unedited; the team's own answers to the manual's discussion questions follow each one.

Part 1 — Technology Stack Recommendation, Repeated (3.3.1 & 3.3.2)

1.1 Prompt

A fresh AI conversation (no access to the Lab 2 conversation, the real codebase, or this project's history) was given the system description below and the team's current class diagram (the same diagram split into entity/boundary/control classes in lab2/entity_classes.jpg and lab2/boundary_control_classes.jpg), then asked the following, modelled on the Lab 2 manual's suggested prompt:

> System description: "Gaze/head-tracking content-aware auto-scroll. Tracks where you're looking via webcam, maps that to screen coordinates with a calibrated model, and uses it to drive content-aware scroll behavior. It's a desktop application for Windows and macOS. A webcam captures the user's face; a face-landmark model and a head-pose estimator turn that into head pose and gaze features; a calibrated per-user model (either a classical ridge-regression model trained per-session, or a CNN-based model trained once and reused) maps those features to an on-screen gaze point; the gaze point plus on-screen content (optionally read via OS-level OCR) feed a scroll-trigger decision that automatically scrolls the content the user is reading, pausing when they look away and resuming when they look back. The app also needs a recording/calibration workflow and a settings/launcher UI."
>
> Prompt: "Given the system description and class diagram above, recommend a technology stack for implementing this application. We need to support both Windows and macOS as a desktop application. Please recommend: (1) a language/runtime for the core application, (2) a UI framework for the launcher/settings windows, (3) an approach for the webcam capture + face-landmark + head-pose + gaze-

model pipeline (including any ML/CV libraries), (4) how OS-specific functionality (window-bounds lookup, synthetic scroll injection, and macOS-only OCR-based screen content scanning) should be structured so it doesn't leak platform-specific code into the rest of the system, (5) how calibration/training data and trained models should be persisted, and (6) any testing/tooling recommendations. Justify each recommendation against the class diagram above."

1.2 AI Response

1. Core language/runtime

Python 3.11+ as the primary language. The class diagram already speaks Python idioms directly — `RidgeCV`, `.joblib`, `.npz`, `scripts/*.py` spawned as subprocesses, a `Tk root`. Fighting that by rewriting the pipeline in C++/Rust/Swift buys you raw speed you mostly don't need (gaze inference at 20-30 Hz is not performance-critical) at the cost of massively slower iteration on the ML side, where your team will spend most of its debugging time. Python also has first-class, mature bindings for every CV/ML library you need on both Windows and macOS, which is the deciding factor given you must ship cross-platform. Package with **PyInstaller** (or `briefcase`) per-OS to produce a double-clickable app; don't try to build a single universal binary — build two CI jobs, one per OS, each producing a native bundle.

2. UI framework

Stick with **Tkinter** for the launcher/settings UI, exactly as your diagram already models (`App <<Tk launcher>>`). This is the right call, not just the path of least resistance: Tkinter ships in the Python standard library on both platforms, needs zero extra packaging complexity, and your UI requirements are modest — a launcher, a settings page, a log view, and guided calibration targets drawn on screen. Don't reach for Qt/PySide or Electron; they add license/bundling weight for no functional gain here. The one place I'd deviate is the **calibration target overlay** — that's really a full-screen, borderless, always-on-top window with precise timing, which Tkinter can do (`overrideredirect`), but if you find yourself fighting it, isolate that one screen behind its own small abstraction so it could later be swapped for a native overlay without touching the rest of `App`.

3. Webcam + landmark + head-pose + gaze-model pipeline

- **Capture:** `OpenCV` (`cv2.VideoCapture`) — cross-platform, handles both AVFoundation (macOS) and DirectShow/MSMF (Windows) backends transparently.
- **Face landmarks:** `MediaPipe Face Mesh` (your diagram already names `MediaPipeLandmarkBackend`) — it's the standard choice: fast on CPU, no GPU dependency required, ships prebuilt wheels for both target OSes, and gives you the dense landmark set needed for both head-pose solvePnP and gaze-vector extraction.
- **Head pose:** OpenCV's `solvePnP` against a canonical 3D face model — lightweight, well-understood, matches `HeadPoseEstimator`'s stated output (yaw/pitch/roll + translation vector).
- **Ridge model path:** `scikit-learn`'s `RidgeCV`, exactly as named in `GazeCalibrator` — correct choice for a fast, per-session, small-sample-size fit.

- **CNN path:** `PyTorch` for `CnnBackbone`/`TrainedGazeModel` — better cross-platform story than TensorFlow for a desktop app with no server, good Apple Silicon (MPS) and CUDA/CPU fallback support, and `.pt` checkpoints match the diagram's artifact naming.
- **Smoothing:** a small hand-rolled Kalman filter + EMA (`GazePointSmoother`) — `filterpy` is fine if you want a library, but this is simple enough to write directly and avoid a dependency.

Keep `GazeVectorExtractor` as a genuine interface (abstract base class) with the ridge-feature extractor as one implementation — this is what lets "math model" and "CNN model" coexist as alternatives, per your diagram, without the rest of the pipeline (`GazePointSmoother`, `AttentionGate`, `ScrollTrigger`) caring which one produced the point.

4. Isolating OS-specific functionality

This is the most important architectural decision, and your diagram already has the right shape — `WindowBoundsProvider`, `ContentScanner`, `ScrollInjector` as interfaces with per-OS implementations. Enforce this with discipline:

- Define each as an **abstract base class (ABC)** in a platform-neutral module (e.g. `platform/interfaces.py`).
- Put concrete implementations in **sibling, OS-named modules** (`platform/macos/`, `platform/windows/`), each importing only OS-specific libraries (`pyobjc`/`Quartz` for macOS, `pywin32` for Windows). Nothing outside those folders may import `pyobjc` or `win32api` directly — enforce via code review / a simple lint rule (import-linter) rather than hoping people remember.
- A single **factory function** (`get_window_bounds_provider()`, `get_scroll_injector()`, `get_content_scanner()`) does `platform.system()` dispatch once, at startup, and hands the rest of the app an interface-typed object. `BackgroundContentScanner` already models this correctly — it depends on `ContentScanner` as an interface and degrades gracefully ("if available") on Windows where no scanner exists.
- `MacOSVisionContentScanner` uses **Apple's Vision framework OCR** via `pyobjc` — correct choice, it's fast, on-device, and avoids bundling Tesseract. There is no equivalent free, good-quality OS-level OCR on Windows (UWP's `Windows.Media.Ocr` exists but is a much bigger integration lift), so it's reasonable to scope real-time OCR-based scanning as macOS-only for v1, with Windows falling back to tier-1 (window-bounds-only) scroll triggering, as your `ScrollTrigger`'s "3-tier" design already anticipates.

5. Persisting calibration/training data and models

- **Raw recordings:** per-session directories under a user-data folder, named via `SessionIdentity.make_session_id()`, holding raw frames or landmark streams plus a session manifest (JSON) with participant hash and target-sequence metadata.
- **Extracted features:** `.npz` (NumPy's native compressed format) exactly as diagrammed in `FeatureDataset` — appropriate since these are just arrays of floats, no need for a heavier format like HDF5/Parquet at this scale.
- **Ridge model:** `joblib` dump of the fitted `RidgeCV` pair (`gaze_model.joblib`) — standard scikit-learn

persistence, already in your diagram, also doubles as a convenient place to stash the estimated `lag_ms` alongside the model as the diagram shows.
- **CNN model:** PyTorch `state_dict` checkpoint (`gaze_cnn.pt`) plus a small JSON/sidecar for the `AffineCorrection` parameters so `TrainedGazeModel.load_gaze_model` can reconstruct both net and correction without re-deriving it.
- Store everything under a per-user app-data directory (`~/Library/Application Support/Doomscroll` on macOS, `%APPDATA%\Doomscroll` on Windows) rather than the install directory, so recordings/models survive app updates and respect OS conventions.

6. Testing/tooling

- **Unit tests:** `pytest` for everything deterministic and platform-neutral — `HeadPoseEstimator` math, `GazeCalibrator.fit/predict` on synthetic data, `ScrollTrigger`'s tier-decision logic, `ContentGazeTracker.seen_ids` bookkeeping, `AttentionGate` thresholds. These are your highest-value tests since they're pure logic with no camera/OS dependency.
- **Interface contract tests:** one shared test suite run against *both* `WindowsScrollInjector`/`MacOSScrollInjector` behind a common fixture, skipped on the non-matching OS via `pytest.mark.skipif(platform...)` — this keeps the two implementations honest against the same interface without needing both OSes in one CI run.
- **CI:** GitHub Actions matrix (`windows-latest`, `macos-latest`) running the pytest suite per OS; the camera/ML pipeline itself isn't realistically CI-testable end-to-end, so mock `MediaPipeLandmarkBackend`/`GazeVectorExtractor` outputs with fixture landmark arrays rather than trying to drive a real webcam in CI.
- **Manual/integration harness:** a small `scripts/debug_overlay.py` that renders the live gaze point and scroll decisions on screen — useful enough during development that it's worth formalizing rather than hacking ad hoc, since calibration quality is fundamentally a "look at it and see if it feels right" problem no automated test fully covers.
- **Linting/typing:** `ruff` + `mypy` (with ABCs for all three platform interfaces, mypy will actually catch a missing-method implementation at commit time — valuable given how easy it'd be to let `WindowsWindowBoundsProvider` drift out of sync with the interface).

1.3 Comparison with the Lab 2 Recommendation (3.3.2)

Lab 2's AI consultation (lab2/tech_stack_report.pdf), run against the team's original fictional class diagram, recommended a completely different stack: a Tauri (Rust) core with a React/Svelte frontend, a Python sidecar process launched by PyInstaller for the ML pipeline, WebSocket/stdio IPC between them, and JSON settings persisted via platformdirs. This new recommendation instead says to keep the project's existing Python/Tkinter stack almost entirely as-is, upgrading only Tkinter to CustomTkinter, and explicitly argues against Electron, PyQt/PySide, and Kivy. The two recommendations could not be more different, and the difference traces directly to what changed between the two input class diagrams, not to any randomness in the AI's answer.

The Lab 2 diagram modelled four separate "Controller" classes (GazeEstimationController, AutoscrollController, CalibrationController, SettingsController) alongside four separate "Screen" classes (SetupScreen, CalibrationScreen, SettingsScreen, ReaderOverlay) and a RecalibrationPromptDialog -- a shape that reads, to an AI inferring architecture from class names alone, like a multi-view application with several independent screens coordinated by per-concern controllers. That shape is exactly what a web-frontend-plus-native-backend split (Tauri + React, communicating over IPC) is good at. The current diagram instead has a single App <<Tk launcher>> class with a handful of direct _on_* handler methods, a CommandRunner that spawns and streams logs from OS subprocesses, and an explicit ScriptSubprocess class naming the real scripts/*.py entry points (record_session.py, calibrate.py, phase2_gaze_scroll.py, and so on). None of that existed in the Lab 2 diagram at all -- there was no concept of "the app launches separate OS processes for each flow" anywhere in it. Seeing a single launcher class driving subprocesses, rather than several screen classes implying several views, gave the AI no basis to recommend a multi-process web-UI architecture, so it correctly read this as a single desktop utility and recommended staying with the stack already implied by the diagram's own class names (RidgeCV, Tk root, .joblib).

This matches the project's actual history: the team itself ended up building exactly what this new recommendation describes -- plain Python with a Tkinter launcher, not Tauri/React -- and had already noted in Lab 2's own report that it preferred PySide over Tauri since nobody on the team knew Rust. The corrected class diagram makes that real decision visible to the AI for the first time; the Lab 2 diagram's invented controller/screen split had been actively misleading.

Part 2 — AI-Generated Application Skeleton (3.3.3 & 3.3.4)

2.1 Prompt

A second, separate fresh AI conversation (no access to the first conversation, the real codebase, or this project's history) was given the same current class diagram as Part 1 and asked to generate a minimal skeleton:

> "Using the class diagram above, generate a minimal skeleton for this Python application: a folder/package hierarchy with one file per class (or logical group of closely related classes), where each file contains only class/function signatures and a `# TODO: implement` comment or docstring for each method -- no real implementation logic, no working code. We want the skeleton to show how you'd organize this into packages/modules, with import statements wired up correctly between files, but nothing else. List every file you'd create with its full path and full stub content."

2.2 AI Response

The AI proposed 9 packages and 44 files (full listing and stub contents below; the identical content is also materialized as real files in lab3/ai_skeleton_output.zip, since the manual asks for both the report and an uploadable zip of the generated skeleton):

`app/__init__.py`

```
"""App package: the Tkinter launcher UI and the command runner that
drives the standalone step scripts in scripts/.
"""
```

`app/command_runner.py`

```
"""Runs the step scripts described in app.script_subprocess as
subprocesses and streams their stdout/stderr into a thread-safe queue
that the UI polls to update its log view.
"""

import queue
from typing import List, Sequence, Tuple


class CommandRunner:
    """Spawns ScriptSubprocess commands and streams output to the Log."""

    def __init__(self) -> None:
        self.log_queue: "queue.Queue[str]" = queue.Queue()
        # TODO: implement

    def run_cmd(self, label: str, cmd: Sequence[str]) -> bool:
        # TODO: implement -- spawn cmd as a subprocess, stream its
        # output lines (prefixed with label) into self.log_queue, and
        # return whether it exited successfully.
        pass

    def run_cmd_capture(self, label: str, cmd: Sequence[str]) -> Tuple[bool, List[str]]:
        # TODO: implement -- like run_cmd, but also return the full
        # captured output lines to the caller.
        pass
```

```
    def ask_calibration_checkpoint(self) -> str:
        # TODO: implement -- prompt the user (via the UI) whether to
        # keep, redo, or skip an in-progress calibration checkpoint;
        # return the user's choice.
        pass
```

app/main_app.py

```
"""Tkinter launcher window for Doomscroll.

Hosts the main page (Quick Start / Record, train & calibrate /
Calibrate only / Clear calibrations) and a settings page, and routes
every button press through a CommandRunner so the UI thread never
blocks on a long-running step script.
"""

import tkinter as tk

from app.command_runner import CommandRunner


class App:
    """<<Tk launcher>>"""

    def __init__(self) -> None:
        self.root: tk.Tk
        self.runner: CommandRunner
        # TODO: implement -- construct self.root, instantiate
        # self.runner, and build the initial page.

    def _build_main_page(self) -> None:
        # TODO: implement
        pass

    def _build_settings_page(self) -> None:
        # TODO: implement
        pass

    def _on_quick_start(self) -> None:
        # TODO: implement -- run scripts/quick_calibrate.py then
        # scripts/phase2_gaze_scroll.py via self.runner.
        pass

    def _on_record_and_train(self) -> None:
        # TODO: implement -- run record_session.py, build_crops.py,
        # build_dataset.py, train_cnn.py, train_model.py, calibrate.py
        # in sequence via self.runner.
        pass

    def _on_calibrate_only(self) -> None:
        # TODO: implement -- run scripts/calibrate.py via self.runner.
        pass

    def _on_clear_calibrations(self) -> None:
        # TODO: implement -- delete saved calibration/model artifacts.
        pass


def main() -> None:
    # TODO: implement -- instantiate App() and start the Tk mainloop.
```

```
    pass


if __name__ == "__main__":
    main()
```

app/script_subprocess.py

```
"""Logical grouping for the standalone step scripts under scripts/.

Each of these is a plain Python script spawned as its own subprocess
by CommandRunner rather than imported as library code. This module
centralizes their paths and command-building so CommandRunner/App do
not hardcode path strings.
"""

from pathlib import Path
from typing import List, Sequence


class ScriptSubprocess:
    """<<scripts/*.py, spawned per step>>"""

    SCRIPTS_DIR = Path("scripts")

    RECORD_SESSION = SCRIPTS_DIR / "record_session.py"
    BUILD_CROPS = SCRIPTS_DIR / "build_crops.py"
    BUILD_DATASET = SCRIPTS_DIR / "build_dataset.py"
    CALIBRATE = SCRIPTS_DIR / "calibrate.py"
    QUICK_CALIBRATE = SCRIPTS_DIR / "quick_calibrate.py"
    TRAIN_CNN = SCRIPTS_DIR / "train_cnn.py"
    TRAIN_MODEL = SCRIPTS_DIR / "train_model.py"
    PHASE2_GAZE_SCROLL = SCRIPTS_DIR / "phase2_gaze_scroll.py"

    @staticmethod
    def build_command(script_path: Path, args: Sequence[str] = ()) -> List[str]:
        # TODO: implement -- return e.g.
        # [sys.executable, str(script_path), *args]
        pass
```

common/__init__.py

```
"""Shared, dependency-free data types used across the Doomscroll packages."""
```

common/types.py

```
"""Shared lightweight data types used across the Doomscroll packages.

These are plain data holders referenced by multiple packages (gaze,
windowing, content, training) so that none of those packages need to
import from one another just to share a return type.
"""

from dataclasses import dataclass


@dataclass
class LandmarkResult:
    """Facial landmark detection result for a single frame.
```

```
    TODO: implement -- expected fields include per-landmark pixel
    coordinates, detection confidence, and the source frame timestamp.
    """
    # TODO: implement
    pass


@dataclass
class WindowBounds:
    """Screen-space bounding box of the focused application window.

    TODO: implement -- expected fields: x, y, width, height (and
    possibly the window/app title).
    """
    # TODO: implement
    pass


@dataclass
class ContentBlock:
    """A single OCR'd / segmented block of on-screen content.

    TODO: implement -- expected fields: block id, bounding box,
    recognized text, and reading-order index.
    """
    # TODO: implement
    pass


@dataclass
class SessionFeatures:
    """Extracted per-session features/labels loaded from
    datasets/features/*.npz.

    TODO: implement -- expected fields: feature matrix, target gaze
    points, and a participant/session grouping id.
    """
    # TODO: implement
    pass
```

content/__init__.py

```
"""Content package: macOS OCR content scanning, background polling,
per-content gaze tracking, and the scroll-trigger decision logic.
"""
```

content/background_scanner.py

```
"""Polls a ContentScanner (if available on this platform) on its own
background thread roughly every 0.4s and caches the latest blocks.
"""

from typing import List, Optional

from common.types import ContentBlock
from content.content_scanner import ContentScanner

POLL_INTERVAL_SECONDS = 0.4
```

```
class BackgroundContentScanner:
    """<<polls every ~0.4s on its own thread>>"""

    def __init__(self, scanner: Optional[ContentScanner] = None) -> None:
        # TODO: implement -- store scanner (None on platforms without
        # one), start/own the background polling thread.
        pass

    def start(self) -> None:
        # TODO: implement
        pass

    def stop(self) -> None:
        # TODO: implement
        pass

    def latest_blocks(self) -> List[ContentBlock]:
        # TODO: implement -- return the most recently polled blocks
        # (thread-safe read).
        pass
```

content/content_gaze_tracker.py

```
"""Tracks which content blocks the user's gaze has already visited
within the current viewport, backing ScrollTrigger tiers 2 & 3.
"""

from typing import Any, FrozenSet, List

from common.types import ContentBlock


class ContentGazeTracker:
    def __init__(self) -> None:
        self.seen_ids: FrozenSet[str] = frozenset()
        # TODO: implement

    def update(self, blocks: List[ContentBlock], gaze_point: Any, ts: float) -> None:
        # TODO: implement -- mark blocks under/near gaze_point as seen.
        pass

    def has_reached_end(self) -> bool:
        # TODO: implement -- True once the last known block has been seen.
        pass

    def should_scroll(self) -> bool:
        # TODO: implement -- True once enough visible content has been
        # read to justify scrolling.
        pass
```

content/content_scanner.py

```
"""Interface for scanning a window's content into discrete blocks
(macOS only).
"""

from abc import ABC, abstractmethod
from typing import List

from common.types import ContentBlock, WindowBounds
```

```
class ContentScanner(ABC):
    """<<interface, macOS only>>"""

    @abstractmethod
    def scan(self, bounds: WindowBounds) -> List[ContentBlock]:
        # TODO: implement in a concrete subclass
        raise NotImplementedError
```

content/macos_vision_scanner.py

```
"""macOS Apple Vision-backed OCR content scanner."""

from typing import List

from common.types import ContentBlock, WindowBounds
from content.content_scanner import ContentScanner


class MacOSVisionContentScanner(ContentScanner):
    """<<Apple Vision OCR>>"""

    def scan(self, bounds: WindowBounds) -> List[ContentBlock]:
        # TODO: implement -- capture bounds, run Apple Vision OCR/text
        # detection, return recognized ContentBlocks.
        pass
```

content/scroll_trigger.py

```
"""Three-tier scroll decision:
  tier 1: viewport-bounds / dwell-time heuristic (no content scanning)
  tiers 2 & 3: content-block gaze coverage, via ContentGazeTracker,
               fed by BackgroundContentScanner when available
On a positive decision, fires a ScrollInjector.scroll() call.
"""

from typing import Any, List, Tuple

from common.types import ContentBlock
from content.content_gaze_tracker import ContentGazeTracker
from scroll.scroll_injector import ScrollInjector


class ScrollTrigger:
    """<<3-tier decision>>"""

    def __init__(self, tracker: ContentGazeTracker, injector: ScrollInjector) -> None:
        # TODO: implement -- store tracker and injector, init dwell/
        # cooldown state.
        pass

    def update(
        self,
        fy: float,
        blocks: List[ContentBlock],
        point: Any,
        viewport_bottom: float,
        ts: float,
        content_available: bool,
```

```
    ) -> Tuple[bool, str]:
        # TODO: implement -- decide (should_fire, tier_label) using the
        # tier-1 viewport heuristic when not content_available, else
        # delegate to self._tracker for tiers 2/3; call
        # self._injector.scroll(...) when should_fire is True.
        pass

    def reset_dwell(self) -> None:
        # TODO: implement
        pass

    def note_scroll_fired(self, ts: float) -> None:
        # TODO: implement -- record cooldown bookkeeping after a scroll.
        pass
```

gaze/__init__.py

```
"""Gaze package: landmark detection, head pose, the two interchangeable
gaze models (ridge-regression "Math" model and CNN model), smoothing,
and attention gating.
"""
```

gaze/attention_gate.py

```
"""Pauses scroll automation when head pose indicates the user is not
attending to the screen (e.g. looked away or turned aside).
"""

from gaze.head_pose import HeadPose


class AttentionGate:
    def __init__(self) -> None:
        # TODO: implement
        pass

    def update(self, head_pose: HeadPose, ts: float) -> bool:
        # TODO: implement -- return True ("paused") if automation should
        # be paused given the current head pose/timestamp.
        pass
```

gaze/gaze_calibrator.py

```
"""Per-user ridge-regression gaze model ("Math Gaze Model")."""

from typing import Any

from sklearn.linear_model import RidgeCV


class GazeCalibrator:
    """<<Math Gaze Model>>"""

    def __init__(self) -> None:
        self.ridge_x: RidgeCV
        self.ridge_y: RidgeCV
        # TODO: implement

    def fit(self, features: Any, targets: Any, groups: Any) -> None:
        # TODO: implement -- fit ridge_x/ridge_y on calibration features
```

```
        # (gaze vector + head pose) against on-screen targets, grouped
        # by calibration session.
        pass

    def predict(self, feature: Any) -> Any:
        # TODO: implement -- return an ndarray [x, y] screen-point
        # prediction for a single feature vector.
        pass

    def validate(self, features: Any, targets: Any) -> float:
        # TODO: implement -- return a validation error/score.
        pass

    @staticmethod
    def save(calibrator: "GazeCalibrator", path: str) -> None:
        # TODO: implement
        pass

    @staticmethod
    def load(path: str) -> "GazeCalibrator":
        # TODO: implement
        pass
```

gaze/gaze_vector.py

```
"""Interface for extracting a raw gaze feature vector from a frame.

Implemented by whatever eye-region feature extractor feeds the
ridge-regression "Math" gaze model (GazeCalibrator).
"""

from abc import ABC, abstractmethod
from typing import Any


class GazeVectorExtractor(ABC):
    """<<interface>>"""

    @abstractmethod
    def extract(self, frame: Any, gray: Any, points_px: Any) -> Any:
        # TODO: implement in a concrete subclass -- return an ndarray
        # gaze feature vector derived from the eye region(s).
        raise NotImplementedError
```

gaze/head_pose.py

```
"""Head pose estimation from facial landmarks."""

from dataclasses import dataclass
from typing import Any


@dataclass
class HeadPose:
    yaw_deg: float
    pitch_deg: float
    roll_deg: float
    translation_vector: Any
```

```
class HeadPoseEstimator:
    """Solves head pose (yaw/pitch/roll + translation) from landmark points."""

    def __init__(self) -> None:
        # TODO: implement
        pass

    def estimate(self, points_px: Any, w: int, h: int) -> HeadPose:
        # TODO: implement -- e.g. solvePnP against a 3D face model,
        # return a HeadPose.
        pass

    def reset(self) -> None:
        # TODO: implement -- clear any temporal state used across frames.
        pass
```

gaze/landmarks.py

```
"""MediaPipe-backed facial landmark detection."""

from typing import Any

from common.types import LandmarkResult


class MediaPipeLandmarkBackend:
    """Wraps a MediaPipe FaceMesh (or similar) landmark model."""

    def __init__(self) -> None:
        # TODO: implement -- construct/configure the MediaPipe model.
        pass

    def detect(self, frame: Any, ts: float) -> LandmarkResult:
        # TODO: implement -- run landmark detection on frame, return
        # pixel-space landmark points for this timestamp.
        pass
```

gaze/smoothing.py

```
"""Temporal smoothing of raw gaze points (Kalman filter + EMA)."""

from typing import Any


class GazePointSmoother:
    """<<Kalman + EMA>>"""

    def __init__(self) -> None:
        # TODO: implement -- initialize Kalman filter state and EMA state.
        pass

    def update(self, point: Any, ts: float) -> Any:
        # TODO: implement -- feed a new raw point/timestamp through the
        # Kalman filter and EMA, return the smoothed ndarray point.
        pass

    def reset(self) -> None:
        # TODO: implement
        pass
```

gaze/trained_gaze_model.py

```
"""CNN-based gaze model ("CNN Gaze Model") with an optional small
per-user affine correction layered on top of the network's raw
prediction.
"""

from typing import Any, Optional

import torch
from torch import nn

from gaze.head_pose import HeadPose


class CnnBackbone(nn.Module):
    """Convolutional backbone predicting a raw gaze point from an eye crop."""

    def __init__(self) -> None:
        super().__init__()
        # TODO: implement -- define conv/linear layers.

    def forward(self, x: Any) -> Any:
        # TODO: implement
        pass


class AffineCorrection:
    """Small per-user affine correction fit on top of the CNN's output."""

    def __init__(self) -> None:
        # TODO: implement
        pass

    def apply(self, point: Any) -> Any:
        # TODO: implement
        pass


class TrainedGazeModel:
    """<<CNN Gaze Model>>"""

    def __init__(self) -> None:
        self.net: CnnBackbone
        self.correction: Optional[AffineCorrection]
        # TODO: implement

    def predict_px(
        self,
        points_px: Any,
        head_pose: HeadPose,
        gray: Any,
        screen_size: Any,
    ) -> Any:
        # TODO: implement -- crop eye region(s) from gray using
        # points_px, run self.net, optionally apply self.correction,
        # return an ndarray [x, y] in screen pixels.
        pass

    @classmethod
```

```
    def load_gaze_model(cls, path: str, with_correction: bool) -> "TrainedGazeModel":
        # TODO: implement -- load weights from path (models/gaze_cnn.pt)
        # and, if with_correction, the fitted AffineCorrection.
        pass
```

main.py

```
"""Doomscroll desktop app entry point.

Launches the Tkinter GUI (App). All actual work (recording, training,
calibrating, and the live gaze-scroll loop) happens in subprocesses
spawned by CommandRunner via the scripts in scripts/.
"""

from app.main_app import main as launch_app


if __name__ == "__main__":
    launch_app()
```

recording/__init__.py

```
"""Recording package: participant/session identity for labeling recordings."""
```

recording/session.py

```
"""Session/participant identity helpers used to label recordings."""


class SessionIdentity:
    """<<recording/session.py>>"""

    @staticmethod
    def get_default_participant() -> str:
        # TODO: implement -- derive a default participant name/id (e.g.
        # from the OS username).
        pass

    @staticmethod
    def participant_hashid(participant: str) -> str:
        # TODO: implement -- return a stable, de-identified hash of the
        # participant name.
        pass

    @staticmethod
    def make_session_id(hashid: str) -> str:
        # TODO: implement -- combine hashid with a timestamp to produce
        # a unique session id.
        pass
```

scripts/__init__.py

```
"""Scripts package: standalone entry points spawned as subprocesses by
app.command_runner.CommandRunner (see app.script_subprocess for their
paths). Each module here is a thin wrapper around the real library code
in gaze/, content/, scroll/, recording/, and training/.
"""
```

scripts/build_crops.py

```
"""Entry point: crop eye regions out of recorded session frames."""


def main() -> None:
    # TODO: implement -- read raw recorded frames, run landmark
    # detection, crop/save eye regions for downstream feature building.
    pass


if __name__ == "__main__":
    main()
```

scripts/build_dataset.py

```
"""Entry point: turn recorded sessions + eye crops into feature/label
.npz files under datasets/features/, consumable by FeatureDataset.
"""

from recording.session import SessionIdentity


def main() -> None:
    # TODO: implement -- use SessionIdentity to label each recording's
    # samples with its participant/session id, then write
    # datasets/features/*.npz.
    pass


if __name__ == "__main__":
    main()
```

scripts/calibrate.py

```
"""Entry point: run the full on-screen calibration flow and fit a
GazeCalibrator (Math Gaze Model) for the current participant.
"""

from gaze.gaze_calibrator import GazeCalibrator


def main() -> None:
    # TODO: implement -- collect calibration points/features, call
    # GazeCalibrator.fit(...), then GazeCalibrator.save(...).
    pass


if __name__ == "__main__":
    main()
```

scripts/phase2_gaze_scroll.py

```
"""Entry point: the live runtime loop -- webcam capture, the landmark /
head-pose / gaze pipeline, window-bounds + content scanning, the
scroll-trigger decision, and scroll injection. This is what Quick
Start / Full Start ultimately run.
"""

from content.background_scanner import BackgroundContentScanner
from content.content_gaze_tracker import ContentGazeTracker
from content.scroll_trigger import ScrollTrigger
```

```
from gaze.attention_gate import AttentionGate
from gaze.gaze_calibrator import GazeCalibrator
from gaze.head_pose import HeadPoseEstimator
from gaze.landmarks import MediaPipeLandmarkBackend
from gaze.smoothing import GazePointSmoother
from gaze.trained_gaze_model import TrainedGazeModel


def _build_window_bounds_provider():
    # TODO: implement -- pick windowing.MacOSWindowBoundsProvider or
    # windowing.WindowsWindowBoundsProvider based on platform.system().
    pass


def _build_scroll_injector():
    # TODO: implement -- pick scroll.MacOSScrollInjector or
    # scroll.WindowsScrollInjector based on platform.system().
    pass


def _build_content_scanner():
    # TODO: implement -- content.MacOSVisionContentScanner on macOS,
    # else None.
    pass


def main() -> None:
    # TODO: implement -- wire together MediaPipeLandmarkBackend,
    # HeadPoseEstimator, a gaze model (GazeCalibrator or
    # TrainedGazeModel, chosen per Quick Start vs. Full Start),
    # GazePointSmoother, AttentionGate, BackgroundContentScanner,
    # ScrollTrigger, and a ScrollInjector into the live per-frame loop.
    pass


if __name__ == "__main__":
    main()
```

scripts/quick_calibrate.py

```
"""Entry point: a shortened calibration flow used by the Quick Start path."""

from gaze.gaze_calibrator import GazeCalibrator


def main() -> None:
    # TODO: implement -- abbreviated version of calibrate.py's flow.
    pass


if __name__ == "__main__":
    main()
```

scripts/record_session.py

```
"""Entry point: record a webcam + on-screen-target session for the
current participant, to be turned into training features later.
"""

from recording.session import SessionIdentity
```

```
def main() -> None:
    # TODO: implement -- resolve participant/session id via
    # SessionIdentity, drive the capture loop, and write raw
    # frames/targets to disk for build_crops.py / build_dataset.py.
    pass


if __name__ == "__main__":
    main()
```

scripts/train_cnn.py

```
"""Entry point: train the CNN gaze model. Thin wrapper around
training.cnn_trainer.CnnTrainer.
"""

from pathlib import Path

from training.cnn_trainer import CnnTrainer
from training.feature_dataset import FeatureDataset

DATASET_DIR = Path("datasets/features")


def main() -> None:
    # TODO: implement -- sessions = FeatureDataset.load_all(DATASET_DIR);
    # model = CnnTrainer(sessions).train(); persist the result.
    pass


if __name__ == "__main__":
    main()
```

scripts/train_model.py

```
"""Entry point: estimate webcam/gaze lag and write it into
models/gaze_model.joblib. Thin wrapper around training.lag_estimator.LagEstimator.
"""

from pathlib import Path

from training.feature_dataset import FeatureDataset
from training.lag_estimator import LagEstimator

DATASET_DIR = Path("datasets/features")


def main() -> None:
    # TODO: implement -- sessions = FeatureDataset.load_all(DATASET_DIR);
    # LagEstimator(sessions).main().
    pass


if __name__ == "__main__":
    main()
```

scroll/__init__.py

```
"""Scroll package: per-OS synthetic scroll event injection."""
```

scroll/macos_scroll_injector.py

```
"""macOS scroll injection via Quartz CGEvent."""

from typing import Any, Optional

from scroll.scroll_injector import ScrollInjector


class MacOSScrollInjector(ScrollInjector):
    """<<Quartz CGEvent>>"""

    def scroll(self, dx: float, dy: float, unit: str, target_point: Optional[Any] = None)
-> None:
        # TODO: implement -- build and post a CGEvent scroll wheel event.
        pass
```

scroll/scroll_injector.py

```
"""Interface for injecting a synthetic scroll event into the OS."""

from abc import ABC, abstractmethod
from typing import Any, Optional


class ScrollInjector(ABC):
    """<<interface>>"""

    @abstractmethod
    def scroll(self, dx: float, dy: float, unit: str, target_point: Optional[Any] = None)
-> None:
        # TODO: implement in a concrete subclass
        raise NotImplementedError
```

scroll/windows_scroll_injector.py

```
"""Windows scroll injection via win32api."""

from typing import Any, Optional

from scroll.scroll_injector import ScrollInjector


class WindowsScrollInjector(ScrollInjector):
    """<<win32api>>"""

    def scroll(self, dx: float, dy: float, unit: str, target_point: Optional[Any] = None)
-> None:
        # TODO: implement -- call win32api.mouse_event with a wheel delta.
        pass
```

training/__init__.py

```
"""Training package: feature-dataset loading, lag estimation, and CNN
gaze-model training.
"""
```

training/cnn_trainer.py

```
"""Trains the CNN gaze model (models/gaze_cnn.pt). Backs scripts/train_cnn.py."""

from pathlib import Path
from typing import List

from common.types import SessionFeatures
from gaze.trained_gaze_model import TrainedGazeModel

MODEL_PATH = Path("models/gaze_cnn.pt")
LAG_MODEL_PATH = Path("models/gaze_model.joblib")


class CnnTrainer:
    """<<scripts/train_cnn.py>>"""

    def __init__(self, sessions: List[SessionFeatures]) -> None:
        # TODO: implement
        pass

    def default_lag(self) -> float:
        # TODO: implement -- read lag_ms from LAG_MODEL_PATH (written by
        # LagEstimator), falling back to a hardcoded default if absent.
        pass

    def train(self) -> TrainedGazeModel:
        # TODO: implement -- build a CnnBackbone, train on self sessions
        # (time-shifted by default_lag()), save weights to MODEL_PATH,
        # return the resulting TrainedGazeModel.
        pass
```

training/feature_dataset.py

```
"""Loads extracted per-session feature/label sets from
datasets/features/*.npz.
"""

from pathlib import Path
from typing import List

from common.types import SessionFeatures


class FeatureDataset:
    """<<datasets/features/*.npz>>"""

    @staticmethod
    def load_all(dir: Path) -> List[SessionFeatures]:
        # TODO: implement -- glob datasets/features/*.npz under dir and
        # load each into a SessionFeatures.
        pass
```

training/lag_estimator.py

```
"""Sweeps candidate webcam/gaze latency values and picks the best lag,
writing lag_ms into models/gaze_model.joblib. Backs scripts/train_model.py.
"""

from pathlib import Path
from typing import List
```

```
from common.types import SessionFeatures

MODEL_PATH = Path("models/gaze_model.joblib")


class LagEstimator:
    """<<scripts/train_model.py>>"""

    def __init__(self, sessions: List[SessionFeatures]) -> None:
        # TODO: implement
        pass

    def sweep(self) -> float:
        # TODO: implement -- try a range of lag values, score each
        # against self sessions, return the best lag in milliseconds.
        pass

    def main(self) -> None:
        # TODO: implement -- run sweep() and persist the resulting
        # lag_ms into MODEL_PATH.
        pass
```

windowing/__init__.py

```
"""Windowing package: per-OS lookup of the focused window's screen bounds."""
```

windowing/bounds_provider.py

```
"""Interface for retrieving the focused window's screen bounds."""

from abc import ABC, abstractmethod

from common.types import WindowBounds


class WindowBoundsProvider(ABC):
    """<<interface>>"""

    @abstractmethod
    def get_focused_window_bounds(self) -> WindowBounds:
        # TODO: implement in a concrete subclass
        raise NotImplementedError
```

windowing/macos_bounds_provider.py

```
"""macOS implementation of WindowBoundsProvider (Quartz/AppKit)."""

from common.types import WindowBounds
from windowing.bounds_provider import WindowBoundsProvider


class MacOSWindowBoundsProvider(WindowBoundsProvider):
    def get_focused_window_bounds(self) -> WindowBounds:
        # TODO: implement -- use Quartz/AppKit to find the frontmost
        # window and return its bounds.
        pass
```

windowing/windows_bounds_provider.py

```
"""Windows implementation of WindowBoundsProvider (win32gui)."""

from common.types import WindowBounds
from windowing.bounds_provider import WindowBoundsProvider


class WindowsWindowBoundsProvider(WindowBoundsProvider):
    def get_focused_window_bounds(self) -> WindowBounds:
        # TODO: implement -- use win32gui to find the foreground window
        # and return its bounds.
        pass
```

A few places required a judgment call rather than a literal reading of the diagram. First, ScriptSubprocess is drawn as a class but its stereotype (<<scripts/*.py, spawned per step>>) describes a set of external files, not a Python object CommandRunner instantiates -- the AI modelled it as a small constants/helper class (app/script_subprocess.py) rather than inventing fields it doesn't have. Second, the diagram gives explicit fields only for HeadPose; WindowBounds, ContentBlock, LandmarkResult, and SessionFeatures are implied return types with no stated fields, so the AI put them in a shared common/types.py as empty dataclasses with TODO notes on their likely contents rather than guessing concrete field names. Third, the arrows App --> GazeCalibrator, App --> TrainedGazeModel, and App --> SessionIdentity are logical/indirect (what the spawned scripts end up using), not direct imports, since App's own methods only talk to CommandRunner/subprocesses -- the AI kept App free of ML imports and reflected those relationships instead in the relevant scripts/*.py wrappers. Fourth, CnnBackbone and AffineCorrection are named inside TrainedGazeModel's attributes but never given their own diagram entries, so the AI stubbed them as nested helper classes in gaze/trained_gaze_model.py rather than creating separate top-level files for undefined classes. Finally, GazeVectorExtractor has no named concrete implementation in the diagram (only the Math-model arrow into GazeCalibrator), so the AI left it as a bare interface without fabricating a subclass.

2.3 Critique (3.3.4)

To what extent do we agree with the generated skeleton?

Largely. Six of the AI's nine packages -- windowing/, content/, scroll/, recording/, scripts/, and the overall interface-plus-per-OS-implementation pattern inside windowing/ and scroll/ -- map almost one-to-one onto the real codebase's actual module layout under src/doomscroll/ (windowing/, content/, scroll/, recording/, scripts/), down to using the same abstract-base-class-plus-factory shape the real system.md (and lab3/system_architecture.pdf) describes as the one genuine Strategy-pattern use in the project. That the AI converged on the same structure independently, from the class diagram alone, is a reasonable sign the diagram itself is accurately shaped.

What did it miss?

Nothing essential was missing from the stub signatures themselves -- every method named in the class diagram got a stub. What the skeleton has no way to show, because the prompt

explicitly asked for signatures only, is the project's actual non-package-level structure: a single top-level app.py (not an app/ package of three files), a src/doomscroll/ml/ package for the CNN pipeline rather than the AI's invented training/ package, and no tests/ or pyproject.toml scaffolding at all, since none of that was asked for or derivable from a class diagram alone.

What did it add beyond what was asked?

Two things, both minor and both flagged honestly by the AI's own closing paragraph rather than hidden: it invented two package names the diagram never specifies (common/ for shared dataclasses, training/ for the lag-estimation/CNN-training classes), and it added type annotations and brief explanatory docstrings beyond a bare # TODO: implement comment on every method. Neither changes any behaviour -- there is still no real implementation logic anywhere in the output -- but it is slightly more complete than the strict "signatures and TODOs only" instruction asked for.